

Complete Python Roadmap (Beginner → Advanced)

A structured, end-to-end path to learn Python — with free YouTube channels, websites, and every topic explained with a simple definition + example.

Best FREE Resources

YouTube Channels

Channel	Best For
freeCodeCamp.org	Full Python course (4-6 hr single video), beginner-friendly
Corey Schafer	Best for OOP, decorators, file handling, virtual environments
Programming with Mosh	Clean, structured beginner-to-intermediate course
Telusko	Good for Indian learners, explains concepts slowly
CS Dojo	Simple explanations, great for absolute beginners
Tech With Tim	Projects + intermediate concepts (OOP, GUI, web scraping)
NeuralNine	Intermediate/advanced Python, automation, security
Sentdex	Data Science, Machine Learning with Python
Codebasics (Dhaval Patel)	Python for Data Analysis, Pandas, NumPy

Websites

Website	Use
python.org/doc	Official documentation (always accurate)
W3Schools.com/python	Quick topic reference + "Try it" editor
GeeksforGeeks.org	Topic-wise theory + practice problems
RealPython.com	In-depth articles, best for intermediate/advanced
HackerRank / LeetCode / CodeChef	Practice problems & coding tests

Website	Use
Programiz.com	Beginner tutorials with online compiler
Kaggle Learn	Free micro-courses for Python + Data Science
Exercism.org	Practice with mentor feedback

Roadmap Overview (Suggested Order)

1. Basics & Syntax
2. Data Types & Variables
3. Operators
4. Control Flow (if/else, loops)
5. Strings
6. Data Structures (List, Tuple, Set, Dict)
7. Functions
8. Object-Oriented Programming (OOP)
9. Exception Handling
10. File Handling
11. Modules & Packages
12. Iterators, Generators, Comprehensions
13. Decorators & Closures
14. Regular Expressions
15. Working with Dates & Time
16. Multithreading/Multiprocessing (intro)
17. Databases (SQLite/MySQL basics)
18. Virtual Environments & pip
19. Popular Libraries (NumPy, Pandas, Matplotlib, Requests, Flask/Django intro)
20. Projects & Practice

1. Basics & Syntax

Definition: Python is a high-level, interpreted, general-purpose programming language known for readable syntax (uses indentation instead of `{ }`).

```
python
```

```
print("Hello, World!")    # prints text to the screen
```

Comments

```
python
```

```
# This is a single-line comment
"""
This is a
multi-line comment/docstring
"""
```

2. Variables & Data Types

Definition: A variable is a name that stores a value in memory. Python is dynamically typed — you don't declare the type explicitly.

```
python
```

```
name = "Alex"           # str
age = 25                  # int
height = 5.9             # float
is_student = True        # bool
```

Type checking & conversion

```
python
```

```
print(type(age))         # <class 'int'>
age_str = str(age)        # convert int to string
num = int("10")           # convert string to int
```

3. Operators

Definition: Symbols that perform operations on variables/values.

```
python
```

```
# Arithmetic
print(5 + 2, 5 - 2, 5 * 2, 5 / 2, 5 // 2, 5 % 2, 5 ** 2)

# Comparison
print(5 > 2, 5 == 2, 5 != 2)

# Logical
print(True and False, True or False, not True)
```

4. Control Flow

Definition: Statements that control the order in which code executes.

If-Else

```
python

marks = 75
if marks >= 90:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

For Loop — iterates over a sequence

```
python

for i in range(5):
    print(i)    # 0 1 2 3 4
```

While Loop — repeats while condition is True

```
python

count = 0
while count < 3:
    print(count)
    count += 1
```

Break / Continue

```
python
```

```
for i in range(10):
    if i == 5:
        break          # stops the loop
    if i % 2 == 0:
        continue       # skips this iteration
    print(i)
```

5. Strings

Definition: A string is a sequence of characters enclosed in quotes.

```
python
```

```
s = "Python Programming"
print(s.upper())          # PYTHON PROGRAMMING
print(s.lower())          # python programming
print(s.split(" "))       # ['Python', 'Programming']
print(s.replace("Python", "Java"))
print(s[0:6])             # slicing -> 'Python'
print(len(s))             # length of string

# f-strings (formatted strings)
name = "Alex"
print(f"Hello, {name}!")
```

6. Data Structures

List — ordered, mutable collection

```
python
```

```
fruits = ["apple", "banana", "mango"]
fruits.append("grape")
fruits.remove("banana")
print(fruits[0])          # apple
```

Tuple — ordered, immutable collection

```
python

point = (10, 20)
print(point[0])    # 10
```

Set — unordered collection of unique items

```
python

nums = {1, 2, 2, 3}
print(nums)    # {1, 2, 3}
```

Dictionary — key-value pairs

```
python

person = {"name": "Alex", "age": 25}
print(person["name"])    # Alex
person["city"] = "Chennai"
```

7. Functions

Definition: A reusable block of code that performs a specific task.

```
python

def greet(name, greeting="Hello"):
    """Returns a greeting message"""
    return f"{greeting}, {name}!"

print(greet("Alex"))    # Hello, Alex!
print(greet("Alex", "Hi"))    # Hi, Alex!

# Lambda function (anonymous function)
square = lambda x: x * x
print(square(5))    # 25
```

Definition: A programming paradigm based on "objects" that contain data (attributes) and behavior (methods).

python

```
class Car:
    def __init__(self, brand, speed):  # constructor
        self.brand = brand
        self.speed = speed

    def drive(self):                  # method
        print(f"{self.brand} is driving at {self.speed} km/h")

my_car = Car("Toyota", 120)
my_car.drive()  # Toyota is driving at 120 km/h
```

Inheritance — child class reuses parent class code

python

```
class ElectricCar(Car):
    def __init__(self, brand, speed, battery):
        super().__init__(brand, speed)
        self.battery = battery

tesla = ElectricCar("Tesla", 150, "100kWh")
tesla.drive()
```

Encapsulation, Polymorphism, Abstraction are the other 3 OOP pillars — worth studying with Corey Schafer's OOP playlist.

9. Exception Handling

Definition: A mechanism to handle runtime errors gracefully instead of crashing the program.

python

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
except ZeroDivisionError:
    print("Cannot divide by zero!")
except ValueError:
    print("Invalid input!")
else:
    print("Result:", result)
finally:
    print("Execution finished.")
```

10. File Handling

Definition: Reading from and writing to files stored on disk.

```
python

# Writing to a file
with open("data.txt", "w") as f:
    f.write("Hello, Python!")

# Reading from a file
with open("data.txt", "r") as f:
    content = f.read()
    print(content)
```

11. Modules & Packages

Definition: A module is a `.py` file containing reusable code; a package is a folder of modules.

```
python
```



```
import math
print(math.sqrt(16))      # 4.0

from datetime import date
print(date.today())

# Installing external packages
# pip install requests
import requests
```

12. Iterators, Generators & Comprehensions

Iterator — object that produces items one at a time

```
python

nums = iter([1, 2, 3])
print(next(nums))      # 1
```

Generator — function that yields values lazily (memory efficient)

```
python

def count_up(n):
    for i in range(n):
        yield i

for num in count_up(3):
    print(num)      # 0 1 2
```

List Comprehension — concise way to build lists

```
python

squares = [x**2 for x in range(5)]
print(squares)      # [0, 1, 4, 9, 16]
```

13. Decorators & Closures

Definition: A decorator is a function that wraps another function to extend its behavior

without modifying it.

```
python

def my_decorator(func):
    def wrapper():
        print("Before function runs")
        func()
        print("After function runs")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
```

14. Regular Expressions

Definition: Patterns used to match/search text.

```
python

import re
text = "My email is test@example.com"
match = re.search(r"[\w.]+@[\w.]+", text)
print(match.group())    # test@example.com
```

15. Dates & Time

```
python

from datetime import datetime
now = datetime.now()
print(now.strftime("%Y-%m-%d %H:%M:%S"))
```

16. Multithreading (Intro)

Definition: Running multiple tasks concurrently to save time on I/O-bound operations.

```
python

import threading

def task():
    print("Task running")

t = threading.Thread(target=task)
t.start()
t.join()
```

17. Databases (SQLite basics)

```
python

import sqlite3
conn = sqlite3.connect("mydb.db")
cur = conn.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER, name TEXT)")
cur.execute("INSERT INTO users VALUES (1, 'Alex')")
conn.commit()
```

18. Virtual Environments & pip

Definition: An isolated Python environment so project dependencies don't conflict.

```
bash

python -m venv myenv
myenv\Scripts\activate      # Windows
source myenv/bin/activate   # Mac/Linux
pip install package_name
```

19. Popular Libraries to Learn Next

Library	Purpose
NumPy	Numerical computing, arrays

Library	Purpose
Pandas	Data analysis, dataframes
Matplotlib / Seaborn	Data visualization
Requests	Making HTTP requests / API calls
Flask / Django	Web development
BeautifulSoup / Scrapy	Web scraping
Scikit-learn / TensorFlow	Machine Learning

Example (NumPy):

```
python

import numpy as np
arr = np.array([1, 2, 3])
print(arr * 2)    # [2 4 6]
```

20. Practice & Projects (build these in order)

1. Calculator (CLI)
 2. To-Do list app
 3. Number guessing game
 4. Contact book (using dictionaries + file storage)
 5. Password generator
 6. Weather app (using an API with `requests`)
 7. Web scraper (BeautifulSoup)
 8. Student management system (OOP + SQLite)
 9. Simple Flask web app
 10. Data analysis mini-project with Pandas + Matplotlib
-

✓ Suggested Study Plan

Week	Focus
1	Basics, variables, operators, control flow
2	Strings, lists, tuples, sets, dictionaries
3	Functions, OOP basics
4	Exception handling, file handling, modules
5	Comprehensions, generators, decorators
6	Regex, datetime, virtual environments, mini projects
7-8	Pick a track: Data Science (NumPy/Pandas) OR Web Dev (Flask/Django)
Ongoing	Solve 2-3 problems daily on HackerRank/LeetCode

Tip: Watch a topic on YouTube → immediately write the code yourself → solve 2-3 practice problems on that topic before moving to the next.